

Evaluation / Viva Preparation Guide

Automated Brain Tumor Detection & Classification
using Deep Learning

EE7204 / EC7205 — Image Processing & Computer Vision

Detection & Classification only (no segmentation)

University of Ruhuna — Group 03

This guide addresses the five evaluation criteria:

1. You can explain **every line** of code.
2. **Every member** can explain the **whole pipeline**.
3. **Proper validation** is demonstrated.
4. **Results** are clearly stated.
5. **No AI-generated images/slides** — build slides from the real **reports/** outputs.

Study Part A (everyone) → own one Part B section (per member) → drill Part F together.

Contents

1	The 60-second and 3-minute story (everyone memorises this)	2
2	File-by-file deep dive (split across members)	2
2.1	src/download.py — Data acquisition	2
2.2	src/preprocess.py — Image processing (the most “CV” file — know it cold)	2
2.3	src/augment.py — Data augmentation	3
2.4	src/models.py — Architectures	3
2.5	src/train.py — Training (know the 2-stage logic)	4
2.6	src/tune.py — Hyperparameter tuning (KerasTuner Hyperband)	4
2.7	src/evaluate.py — Metrics & plots	5
2.8	src/app.py — Streamlit UI	5
3	Proper validation (criterion 3 — say this explicitly)	5
4	Results, clearly stated (criterion 4)	5
5	Suggested member ownership (criterion 2)	6
6	Anticipated questions & answers (drill these)	6
7	Demo & slide checklist (criterion 5)	7
8	Be honest about these (raise before being asked)	8

1. The 60-second and 3-minute story (everyone memorises this)

60-second pitch: “We built a Computer-Aided Diagnosis system that classifies a brain MRI into one of four classes — *glioma*, *meningioma*, *pituitary*, *no-tumor*. The image is preprocessed with OpenCV (grayscale, denoising, skull stripping, normalization), then classified by a CNN. We compare a from-scratch Custom CNN against three transfer-learning backbones (VGG16, ResNet50, EfficientNetB0). The best model is **ResNet50 at 85.8% test accuracy**. A Streamlit app lets a user upload a scan and get a prediction with a confidence score.”

The data journey (one sentence per stage)

1. **Acquire** — Kaggle Brain Tumor MRI dataset (`src/download.py`).
2. **Preprocess** — load → grayscale → 3-channel → resize 224×224 → Gaussian filter → median filter → skull strip → min-max normalize (`src/preprocess.py`).
3. **Split** — stratified 70% train / 15% val / 15% test.
4. **Augment** — rotation/flip/zoom/shift/brightness/contrast, *training only* (`src/augment.py`).
5. **Model** — Custom CNN + 3 transfer backbones with a custom head (`src/models.py`).
6. **Train** — Adam + categorical cross-entropy, class weights, early stopping; transfer models use a **2-stage** schedule (`src/train.py`).
7. **Tune (optional)** — KerasTuner Hyperband over head width / dropout / LR / L2 (`src/tune.py`).
8. **Evaluate** — accuracy/precision/recall/F1 + confusion matrix on the **held-out test set** (`src/evaluate.py`).
9. **Deploy** — Streamlit UI (`src/app.py`).

2. File-by-file deep dive (split across members)

2.1 `src/download.py` — Data acquisition

- Uses the **Kaggle API**: `authenticate()` then `dataset_download_files(..., unzip=True)`.
- Organises the download into `data/raw/{glioma, meningioma, pituitary, notumor}`; `verify_dataset()` checks all 4 folders exist.
- **Why a constant dataset id?** reproducibility — one canonical public source (Proposal Gap 3).
- **Talking point:** the repo ships a **1,600-image balanced subset** (400/class); the full set is $\sim 7,023$.

2.2 `src/preprocess.py` — Image processing (the most “CV” file — know it cold)

Pipeline order: **load** → **gray3** → **resize** → **Gaussian** → **median** → **skull-strip** → **normalize**.

Function	Key call	What & why
<code>load_image</code>	<code>cv2.imread</code>	reads BGR uint8 ; raises if <code>None</code> (corrupt/missing).
<code>to_grayscale_rgb</code>	<code>cvtColor + merge</code>	MRI is 1-channel; ImageNet nets need 3 channels — replicate (no false colour).
<code>resize_image</code>	<code>resize(INTER_AREA)</code>	224 is the ImageNet size; INTER_AREA is best for downscaling .
<code>apply_gaussian_filter</code>	<code>GaussianBlur(3,3)</code>	smooths high-frequency/Gaussian noise; <code>sigmaX=0</code> $\Rightarrow \sigma$ from kernel.
<code>apply_median_filter</code>	<code>medianBlur(3)</code>	removes impulse noise while preserving edges (tumor boundaries).
<code>skull_strip</code>	(see below)	removes skull/background so the CNN focuses on brain tissue.
<code>normalize_image</code>	<code>(x-min)/(max-min)</code>	min-max to $[0,1]$ for stable gradients; guards divide-by-zero.

`skull_strip()` step by step (examiners love this):

1. `threshold(BINARY+OTSU)` — **Otsu** auto-picks the brain/background threshold.
2. `morphologyEx(MORPH_CLOSE, ellipse 7x7, iters=3)` — closing (dilate then erode) fills holes in the mask.
3. `findContours(RETR_EXTERNAL, CHAIN_APPROX_SIMPLE)` — outer contours; store only corner points.
4. `max(contours, key=contourArea)` — the **largest contour = the brain**.
5. `drawContours(FILLED) + bitwise_and` — keep brain pixels, zero the rest.
6. `boundingRect + crop`, then `copyMakeBorder` to **pad to square**, then resize to 224.
7. If no contour found $\rightarrow$ return resized original (never crashes).

`prepare_dataset()`: collects from `Training/` and `Testing/` (we re-split ourselves); two `train_test_split` calls with `stratify=` and `random_state=42` $\Rightarrow$ **70/15/15**, class proportions preserved, reproducible. Saves processed images as **lossless PNG** (JPEG would re-add artefacts).

2.3 `src/augment.py` — Data augmentation

- A `keras.Sequential` of random layers, applied **only with** `training=True` (val/test untouched).
- Values (tuned *down* from defaults): rotation $\pm 15^\circ$ (`factor=0.042`), horizontal flip, zoom $\pm 12\%$, shift $\pm 10\%$, brightness $\pm 20\%$, contrast 0.8–1.2.
- **Why augment?** small dataset $\rightarrow$ synthesise plausible variants $\rightarrow$ less overfitting.
- **Why tuned down?** aggressive rotation/zoom destroys the subtle cues separating glioma vs meningioma.

2.4 `src/models.py` — Architectures

`get_preprocess_input(name)` returns the correct preprocessing per backbone: identity for the Custom CNN ($[0,1]$); `vgg16/resnet50/efficientnet.preprocess_input` for the others (they expect $[0,255]$ and do their own scaling) — this is why `train.py` rescales `x*255` first.

Custom CNN: 5 conv blocks $32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$ (each $\text{Conv}(3 \times 3) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}$), then a 1×1 conv reducing $512 \rightarrow 64$, **Flatten**, $\text{Dense}_{512} \rightarrow \text{BN} \rightarrow \text{Dropout}_{0.5}$, $\text{Dense}_{256} \rightarrow \text{BN} \rightarrow \text{Dropout}_{0.5}$, Dense_4 softmax. $\sim 3.35\text{M}$ params.

- **Flatten not GAP?** GAP discards *spatial location*; pituitary (centre-base) vs meningioma (periphery) depend on location.
- **Why 1×1 conv first?** $7 \times 7 \times 512 = 25,088$ flattened would explode the Dense layer; reduce to $7 \times 7 \times 64 = 3,136$.
- **Single (not double) conv per block?** $\sim 1,120$ training images can't support a deeper net (double-conv underfit).

Transfer models: `Backbone(include_top=False, weights='imagenet') → GlobalAveragePooling2D → Dense_{512} (L2=1e-4) → BN → Dropout 0.4 → Dense_{256} → BN → Dropout 0.3 → Dense_4 softmax.`

- **Stage 1:** every backbone layer `trainable=False` (only the head learns).
- **Stage 2 (`fine_tune=True`):** unfreeze the last 8 / 33 / 50 layers (VGG16/ResNet50/EfficientNet), **except BatchNorm** (keep ImageNet statistics).
- `base(inputs, training=False)` keeps backbone BN in inference mode during head training.

2.5 src/train.py — Training (know the 2-stage logic)

- `_make_dataset`: `image_dataset_from_directory` with a **fixed class_names order** (glioma, meningioma, notumor, pituitary) so label indices are identical in train/eval/app — critical. Pipeline: $/255 \rightarrow (\text{train}) \text{augment} \rightarrow \text{clip}(0,1) \rightarrow (\text{transfer}) \text{preprocess_fn}(x*255) \rightarrow \text{prefetch}$. *Why clip?* `RandomBrightness` can exceed $[0, 1]$.
- `_compute_class_weights`: sklearn 'balanced' — rarer classes weighted higher (robust on the full imbalanced set).
- **Callbacks:** `EarlyStopping` (monitors `val_loss`, `restore_best_weights`), `ModelCheckpoint`, `ReduceLROnPlateau` ($\times 0.5$, patience 3), and `TensorBoard`.
- **Stage 1:** Adam, loss `CategoricalCrossentropy(label_smoothing=0.1)`, with class weights (Custom CNN: lr $5e-4$, ≤ 80 ep; transfer: lr $1e-3$, ≤ 30 ep).
- **Stage 2 (transfer):** `load_model(stage1_best)` → unfreeze top layers → recompile Adam($1e-5$) → fit ≤ 30 .
- **Safety check:** if Stage-1 `val_loss` was lower, it **reverts** to the Stage-1 model.

2.6 src/tune.py — Hyperparameter tuning (KerasTuner Hyperband)

- Searches head width (256/512/1024, 128/256), dropout (0.2–0.6 / 0.1–0.5), learning rate ($1e-3/5e-4/1e-4$), L2 ($1e-5/1e-4/1e-3$). The conv/backbone is fixed.
- **Hyperband** = run many configs briefly, keep winners, give them more epochs (a tournament) — far cheaper than grid search.
- Optimises `val_accuracy` **on the validation split** — exactly the Proposal's "use validation to tune hyperparameters."
- Retrains the best config with the same 2-stage strategy, saving the standard artefacts so evaluation/UI use it unchanged.

2.7 src/evaluate.py — Metrics & plots

- Loads the test set with `shuffle=False` (predictions align with labels), then `predict` and `argmax`.
- sklearn metrics: `weighted accuracy`, `precision`, `recall`, `f1_score`, plus `classification_report` and `confusion_matrix`.
- Saves confusion matrices, training curves (stage1+stage2, with a Stage-2 marker), `model_comparison.png/csv`, and `classification_reports.json`.

2.8 src/app.py — Streamlit UI

Loads a cached model, uploads an image, shows the **6 preprocessing stages**, applies the *same* preprocessing as training, and shows class + confidence + a per-class bar chart — proving the model works on a single unseen image (Proposal: “demonstration on unseen test images”).

3. Proper validation (criterion 3 — say this explicitly)

1. **Data split** — stratified **70/15/15** (`random_state=42`). Test set is **held out** and used *only once*; validation drives early stopping / LR / tuning. No test leakage.
2. **Generalisation guards** — augmentation, Dropout (0.3–0.5), L2 (1e-4), BatchNorm, label smoothing (0.1), **EarlyStopping with restore_best_weights**.
3. **Class imbalance** — balanced class weights (robust on the full dataset).
4. **Fair comparison** — all 4 models share the same split, preprocessing, augmentation, and head $\Rightarrow$ differences are due to the **backbone**.
5. **Multi-model cross-check** — all 4 models show the *same* error pattern (glioma $\leftrightarrow$ meningioma) $\Rightarrow$ a real data property, not a bug.
6. **Metrics** — accuracy, precision, recall, F1, **per-class** report, and confusion matrix (accuracy alone hides per-class failure).
7. **Reproducibility** — fixed seeds, public dataset, one-command pipeline.

Honesty point (graders like this): our test accuracies (84–86%) sit a few points below the 90%+ literature figures *because we use the 1,600-image subset*, not the full $\sim 7k$ set. We state this limitation and the fix (full data / GPU).

4. Results, clearly stated (criterion 4)

Test set: 240 images (60 per class). Weighted metrics:

Model	Accuracy	Precision	Recall	F1
ResNet50 (best)	85.83%	86.52%	85.83%	85.66%
VGG16	84.58%	86.08%	84.58%	84.53%
EfficientNetB0	83.75%	84.47%	83.75%	83.30%
Custom CNN	71.67%	70.03%	71.67%	69.82%

Per-class (ResNet50): notumor F1 ≈ 0.96 , pituitary 0.95, meningioma 0.78, glioma 0.74.

Reading the confusion matrix: rows = true class, columns = predicted; the **diagonal** = correct, each row sums to 60. The off-diagonal hot-spot is **glioma $\leftrightarrow$ meningioma** (shared texture/intensity — the clinically hard pair). **notumor** and **pituitary** are near-perfect.

Conclusions:

1. Transfer learning beats the from-scratch CNN by ~13–14 points — ImageNet features help on small data.
2. **ResNet50 wins**, matching the Proposal (Gap 1: modern residual backbone over heavy VGG / from-scratch).
3. EfficientNetB0 is nearly as good with far fewer parameters (4.8M vs 24.8M) $\Rightarrow$ best for deployment.

Figures to show (real outputs in reports/, *not* AI generated): `confusion_matrix_resnet50.png`, `model_comparison.png`, `overall_metrics_comparison.png`, `per_class_f1_comparison.png`, `training_curves_resnet50.png`.

5. Suggested member ownership (criterion 2)

Everyone must know the *whole* pipeline, but each leads a part for depth. Practise by **swapping** (member 2 explains member 1's code, etc.).

Member	Leads	Must still explain
1	Data: <code>download.py</code> + <code>preprocess.py</code> (esp. skull stripping)	everything
2	Models: <code>models.py</code> (Custom CNN + transfer heads)	everything
3	Training/validation: <code>train.py</code> + <code>tune.py</code> (2-stage, callbacks)	everything
4	Evaluation + UI: <code>evaluate.py</code> , <code>app.py</code> , results	everything

6. Anticipated questions & answers (drill these)

Image processing

Q: Why median AND Gaussian filters?

A: Gaussian kills high-frequency/Gaussian noise but blurs edges; median kills impulse noise and *keeps* edges. Together: clean image, sharp tumor boundaries.

Q: Why Otsu, not a fixed threshold, for skull stripping?

A: Otsu adapts per image (different scanners/brightness); a fixed value fails on darker/brighter scans.

Q: Why pad to square before resizing?

A: Resizing a non-square crop to 224×224 would stretch the brain and distort tumor shape.

Modelling

Q: What is transfer learning?

A: Reuse ImageNet-pretrained conv features, replace the classifier head, train on our 4 classes — works with little data.

Q: Why freeze then fine-tune (2 stages)?

A: Unfreezing immediately lets large random-head gradients destroy pretrained weights. Train the head first, then fine-tune gently at lr $1e-5$.

Q: Why keep BatchNorm frozen during fine-tuning?

A: BN holds ImageNet mean/variance; updating it on a tiny batch destabilises training.

Q: Custom CNN: Flatten vs Global Average Pooling?

A: GAP discards spatial location; pituitary/meningioma depend on location, so we Flatten (after a 1×1 conv to limit parameters).

Training / validation**Q: Difference between validation and test set?**

A: Validation tunes/monitors (early stopping, LR, HP search); the test set is touched *once* for the final unbiased number.

Q: What does early stopping do?

A: Stops when `val_loss` stops improving for `patience` epochs and *restores the best weights* — prevents overfitting.

Q: What is label smoothing?

A: Soft targets (0.9 instead of 1.0) — the model is less over-confident and generalises better.

Q: Is there data leakage?

A: No — the split is on raw images before any model sees them; the test set is never used for tuning.

Results**Q: Why not 90%+ like the papers?**

A: We use the 1,600-image subset; the full $\sim 7k$ dataset + GPU is expected to exceed 90%.

Q: Which class is hardest and why?

A: Glioma vs meningioma — overlapping appearance; visible as the off-diagonal in every confusion matrix.

Q: Why report F1, not just accuracy?

A: Accuracy hides per-class failure; F1 balances precision and recall per class.

“Gotcha” checks**Q: Where are the class labels ordered?**

A: A fixed `CLASS_NAMES` list shared by train/evaluate/app — they must match or predictions are mislabelled.

Q: Why divide by 255 then multiply by 255 again for transfer models?

A: We normalise to $[0, 1]$ for a uniform pipeline, then each backbone's `preprocess_input` wants $[0, 255]$ and applies its own scaling — so we convert back.

7. Demo & slide checklist (criterion 5)

- **Make slides yourself.** Use *your own* plots from `reports/` (genuine experimental outputs, not AI art) and screenshots of the running app.
- **Live demo:** `streamlit run src/app.py` → upload one test image per class → show preprocessing stages + prediction + confidence. Keep a screen-recording as backup.

- **Slide order:** Problem → Objectives → Dataset → Preprocessing (skull-strip before/after) → Augmentation → Models (table) → Training/validation → Results + confusion matrix → Best model & conclusion → Limitations & future work → Live demo.
- Keep a one-line “who built what” slide for criterion 2.

8. Be honest about these (raise before being asked)

- The repo ships a **subset** (1,600 images); README §4 states this.
- Trained weights are *not* committed (`models/` is gitignored) — train to reproduce; the committed `reports/` come from a prior full run (internal consistency verified).
- Hyperparameters were chosen **manually**; `src/tune.py` adds a proper automated search for the HPC.

Good luck — know the “why”, not just the “what”.